

# CS214-Data Structure

Lecturer: Dr. Salwa Osama

Linked Stack and Queue

# Agenda

- Difference between linked and array based.
- Types of linked list
- Implement stack based on linked concept
  - Build stack based on linked list library
  - Build stack from scratch
- Implement queue based on linked concept
  - Build queue based on linked list library
  - Build queue from scratch

| Arrays                                                                                          | Linked list                                                                                             |
|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Fixed size: Resizing is expensive                                                               | Dynamic size                                                                                            |
| Insertions and Deletions are inefficient: Elements are usually shifted                          | Insertions and Deletions are efficient: No shifting                                                     |
| Random access i.e., efficient indexing                                                          | No random access<br>→ Not suitable for operations requiring accessing elements by index such as sorting |
| No memory waste if the array is full or almost full; otherwise may result in much memory waste. | Since memory is allocated dynamically(acc. to our need) there is no waste of memory.                    |
| Sequential access is faster [Reason: Elements in contiguous memory locations]                   | Sequential access is slow [Reason: Elements not in contiguous memory locations]                         |

## Difference between based array and based linked list

# Types of Linked List

- There are three common types of Linked List.
  1. Singly Linked List
  2. Doubly Linked List
  3. Circular Linked List

# Singly Linked List

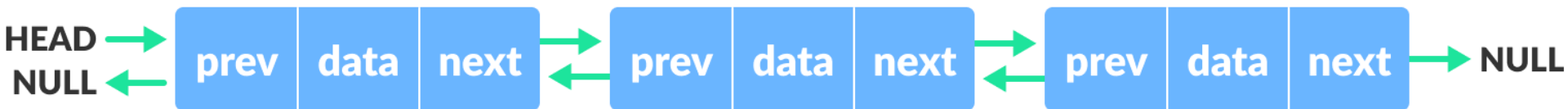
- It is the most common. Each node has data and a pointer to the next node.



```
struct node {  
    int data;  
    struct node *next;  
}
```

# Doubly Linked List

- We add a pointer to the previous node in a doubly-linked list. Thus, we can go in either direction: forward or backward.



```
struct node {  
    int data;  
    struct node *next;  
    struct node *prev;  
}
```

# Circular Linked List

- A circular linked list is a variation of a linked list in which the last element is linked to the first element. This forms a circular loop.



- A circular linked list can be either singly linked or doubly linked.
  - for singly linked list, next pointer of last item points to the first item
  - In the doubly linked list, prev pointer of the first item points to the last item as well.

# Your Mission

- Implement six operations of the circular single linked list
- Implement six operations of the double linked list.





Linked Stack

# Linked Stack Implementation

```
typedef char EntryType;

typedef struct{
    EntryType    info;
    Node    *next;
}Node;

typedef Node * StackType;
```

# STACK Implementation

- Initialization:

Pre: None.

Post: The stack is initialized to be empty.

```
void CreateStack(StackType *s){  
    *s = NULL ;  
}
```

# STACK Implementation

**Stack empty operation:**

**Pre:** The stack is initialized.

**Post:** If the stack is empty (1) is returned. Otherwise (0) is returned.

```
int StackEmpty(StackType s){  
    return (s==NULL);}
```

# STACK Implementation

**Stack full operation:**

**Pre:** The stack is initialized.

**Post:** If the stack is full (1) is returned. Otherwise (0) is returned.

```
int Stackfull(StackType  s){  
    return (0);}
```

# STACK Implementation

## ○ Push operation:

Pre: The stack is initialized and is not full.

Post: Item is added to the top of the stack.

```
void Push(EntryType item, StackType *s){  
    Node *p = (Node *)malloc(sizeof(Node));  
    p->info = item;    p->next = *s;  
    *s = p;  
}
```

# STACK Implementation

## ○ Pop operation:

Pre: The stack is initialized and is not empty.

Post: The top element of the stack is removed from it and is assigned to item.

```
void Pop(EntryType *item, StackType*s){  
    *item = *s->info;  
    Node *p = *s;  
    *s= *s->next;  
    free(p);  
}
```

# Linked Stack Implementation

## ○ Clear operation:

Pre: The stack is initialized.

Post: the stack is cleared to be empty.

```
void ClearStack(StackType *s){
```

```
    Node *q;
```

```
    while(*s){
```

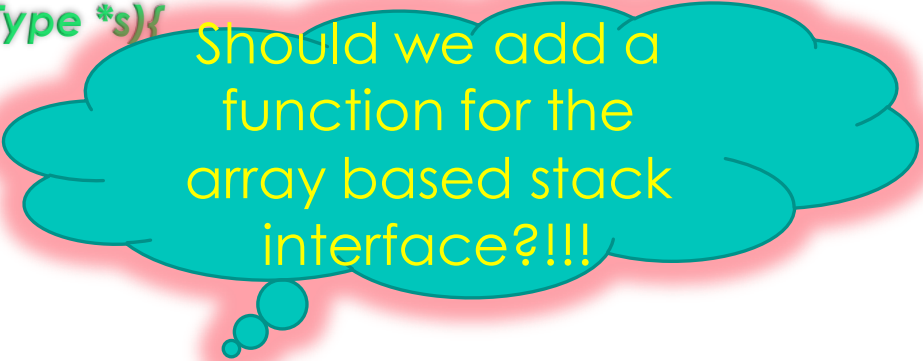
```
        q = *s;
```

```
        *s = *s ->next;
```

```
        free (q);
```

```
    }
```

```
}
```



Should we add a function for the array based stack interface?!!!





# Linked Queue

# Linked Queue Implementation

```
typedef char EntryType;
typedef struct{
    EntryType    info;
    Node    *next;
}Node;
typedef struct{
    Node    *front;
    Node    *rear
} QueueType;
```

# Queue Implementation

- Initialization operation:

Pre: None.

Post: The queue is initialized to be empty.

```
void CreateQueue(QueueType *q){  
q->front= NULL;  
q->rear = NULL;  
}
```

# Queue Implementation

Empty operation:

Pre: The queue is initialized.

Post: If the queue is empty (1) is returned. Otherwise (0) is returned.

```
int QueueEmpty(QueueType q){  
    return (q->front == NULL);}
```

Full operation:

Pre: The queue is initialized.

Post: If the queue is full (1) is returned. Otherwise (0) is returned.

```
int QueueFull(QueueType q){  
    return 0;}
```

# Queue Implementation

- Enqueue operation:

Pre: The queue is initialized and is not full.

Post: Item is added to the end of the stack.

```
void Enqueue(EntryType item, QueueType *q){  
    Node *p = (Node *)malloc(sizeof(Node));  
    p->info = item;    p->next = NULL;  
    if (!q->rear)      q->front = p;  
    else               q->rear->next=p;  
    q->rear=p;  
}
```

# Queue Implementation

## ○ Dequeue operation:

Pre: The Queue is initialized and is not empty.

Post: The front element of the Queue is removed from it and is assigned to item.

```
void dequeue (Entry *item, QueueType *q){  
    *item = q->front->info;  
    Node *p = q->front;  
    q->front = q->front->next;  
    free(p);  
    if (!q->front)    q->rear = NULL;  
}
```

# Linked Queue Implementation

## Clear operation:

Pre: The Queue is initialized.

Post: the Queue is cleared to be empty.

```
void ClearQueue(QueueType *q){
```

```
Node *p;
```

```
while(q->front){
```

```
    p=q->front
```

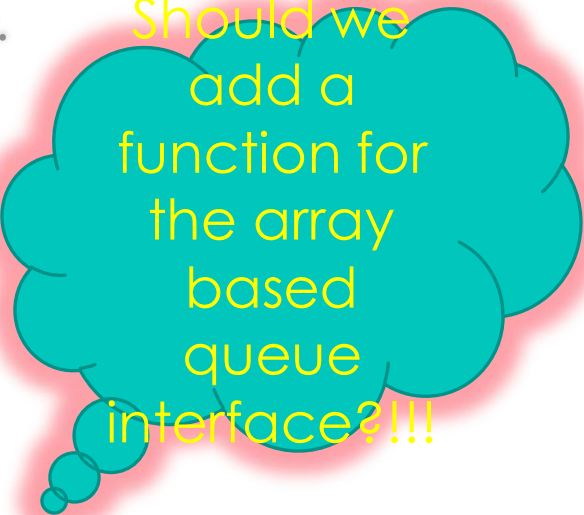
```
    q->front=q->front->next;
```

```
    free(p);
```

```
}
```

```
q->rear = NULL;
```

```
}
```



Should we  
add a  
function for  
the array  
based  
queue  
interface?!!!

